TASK 12

SPARK STREAMING AND KAFKA

Artificial Intelligence Infrastructure course

AUTHOR: MICHAEL O'SHEA (2025244317) & MOHAMMED ABDELQADER (2025179643)

Introduction

This assignment integrates Apache Kafka as a real-time data transport platform with Apache Spark Structured Streaming as the stream-processing layer. The practical objective is to deploy Kafka and Spark, modify the producer to continuously publish aircraft state data and modify the consumer to process the Kafka stream.

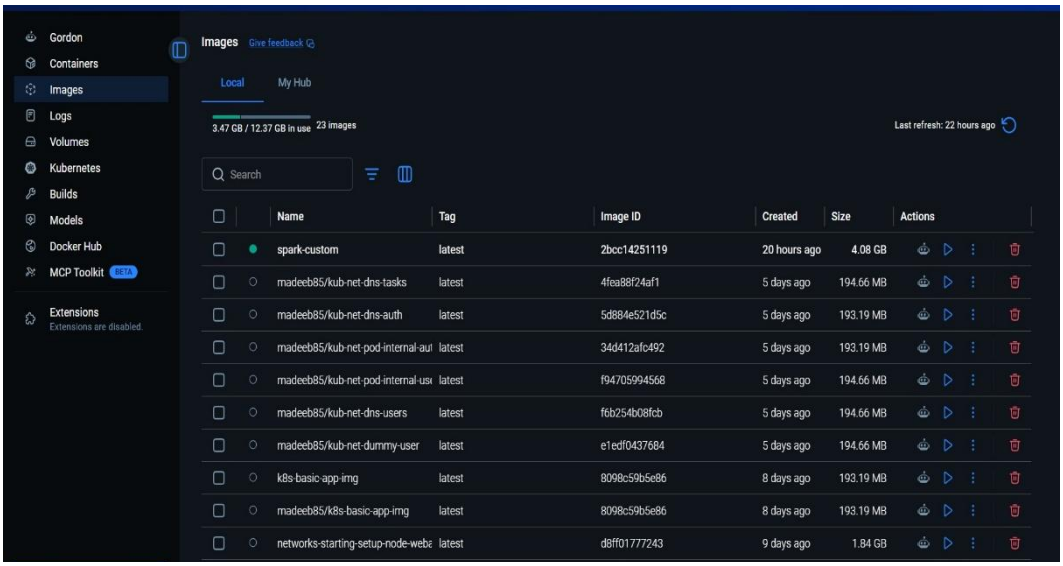
1.Steps Followed

The steps followed were:

1) We chose Option 2, which was to have everything running on PC1 (or PC2) within the same Docker network.

2) Downloaded the code for mod17-SparkStreaming and mod13-Kafka from GitHub.

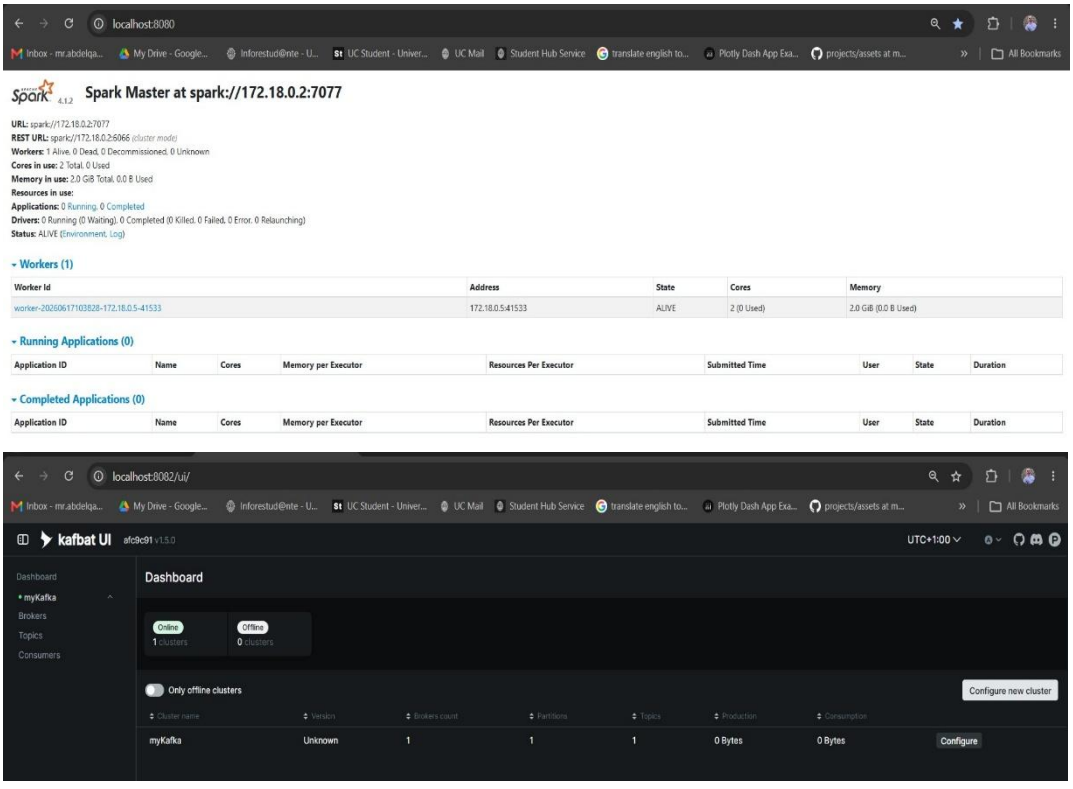
3) Checked the Docker images to determine whether the Spark-custom image needed to be rebuilt. We didn't need to rebuild as it was already there.



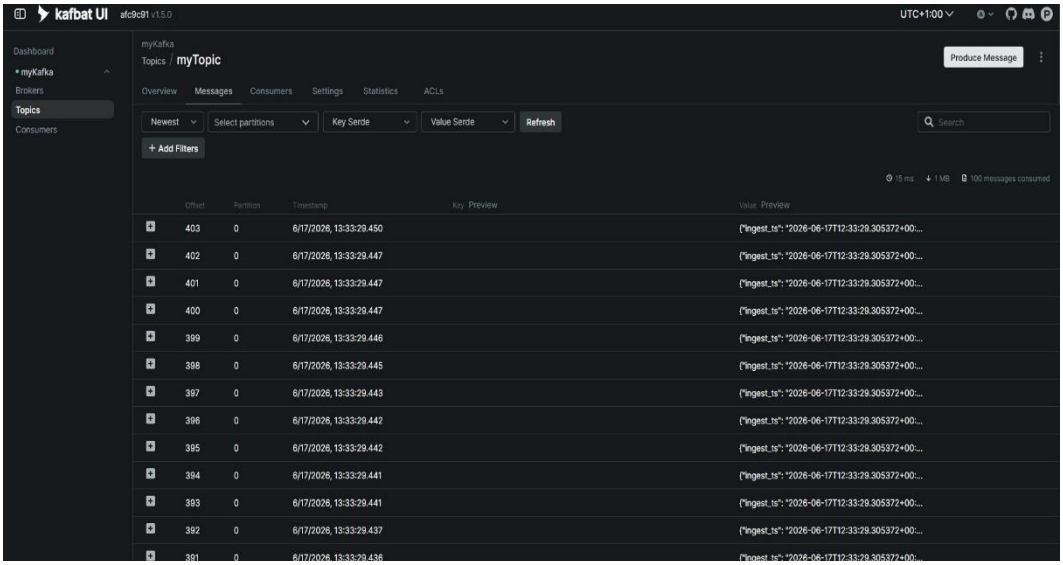
4) Performed the deployment: In this step we merged the Docker Compose for Kafka and Docker Compose for Spark, we took care to make sure that no conflicting entries i.e. Ports configurations. To see the full file configuration, see **Annex 1**.

```yaml
# Merged Docker Compose file: Kafka + Kafka UI + Apache Spark
# Spark containers can reach Kafka using: kafka:29092
# From your host machine, Kafka is reachable on: localhost:9092

services:
  kafka:
    container_name: kafka
    image: apache/kafka:4.3.0
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: "CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,PLAINTEX
      KAFKA_ADVERTISED_LISTENERS: "PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092"
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
      KAFKA_PROCESS_ROLES: "broker,controller"
      KAFKA_NODE_ID: 1
      KAFKA_CONTROLLER_QUORUM_VOTERS: "1@kafka:29093"
      KAFKA_LISTENERS: "PLAINTEXT://kafka:29092,CONTROLLER://kafka:29093,PLAINTEXT_HOST://0.0.
      KAFKA_INTER_BROKER_LISTENER_NAME: "PLAINTEXT"
      KAFKA_CONTROLLER_LISTENER_NAMES: "CONTROLLER"
      KAFKA_LOG_DIRS: "/tmp/kafka-combined-logs"
      CLUSTER_ID: "MkU3OEVBNTcwNTJENDM2Qk"
    volumes:
      - ./kafka-config:/opt/kafka/config/
      - ./kafka-data:/tmp/kafka-combined-logs
    networks:
      - aii-network

  kafbat-ui:
    container_name: kafbat-ui
```

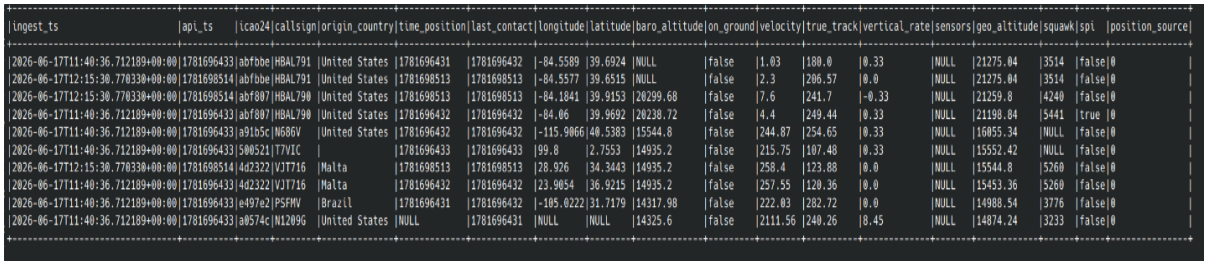
5) Verified that the Spark UI and Kafka UI were reachable.



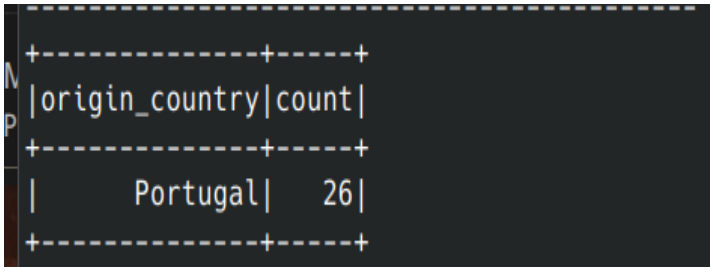
6) Modify the code of the streaming producer: we modified the Producer.py source code to read the API and put the result of that read into Kafka topic. However, add a batching mechanism because the unbatched data exceeded the maximum message size. For the full source code, see **Annex 2**.



7) Modify the code of the streaming consumer: multiple changes needed for the consumer these can be seen in detail in Annex 3. However, the highlights are:
   1) Update the code to read from the Kafka topic
   2) Have the correct settings to connect to Kafka.
   3) Added queries to retrieve requested information.

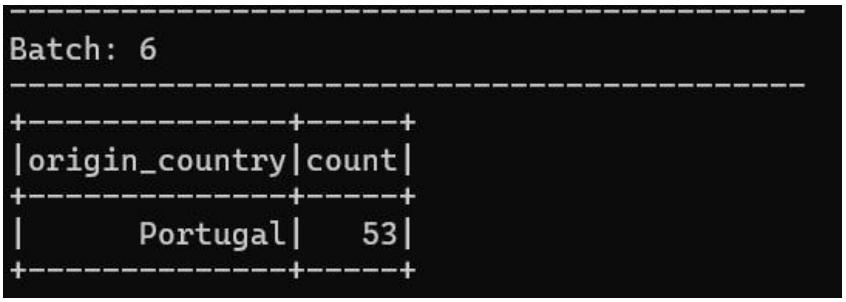


ingest_ts	api_ts	icao24	callsign	origin_country	time_position	last_contact	longitude	latitude	baro_altitude	on_ground	velocity	true_track	vertical_rate	sensors	geo_altitude	squawk	spi	position_source
2026-06-17T11:40:36.712189+00:00	1781696433	abfbbe	HBAL791	United States	1781696431	1781696432	-84.5589	39.6924	NULL	false	1.03	180.0	0.33	NULL	21275.04	3514	false	0
2026-06-17T12:15:30.770330+00:00	1781698514	abfbbe	HBAL791	United States	1781698513	1781698513	-84.5577	39.6515	NULL	false	2.3	206.57	0.0	NULL	21275.04	3514	false	0
2026-06-17T12:15:30.770330+00:00	1781698514	abf807	HBAL790	United States	1781698513	1781698513	-84.1841	39.9153	20299.68	false	7.6	241.7	-0.33	NULL	21259.8	4240	false	0
2026-06-17T11:40:36.712189+00:00	1781696433	abf807	HBAL790	United States	1781696432	1781696432	-84.06	39.9692	20238.72	false	4.4	249.44	0.33	NULL	21198.84	5441	true	0
2026-06-17T11:40:36.712189+00:00	1781696433	a91b5c	N686V	United States	1781696432	1781696432	-115.9866	40.5383	15544.8	false	244.87	254.65	0.33	NULL	16055.34	NULL	false	0
2026-06-17T11:40:36.712189+00:00	1781696433	500521	T7VIC		1781696433	1781696433	99.8	2.7553	14935.2	false	215.75	107.48	0.33	NULL	15552.42	NULL	false	0
2026-06-17T12:15:30.770330+00:00	1781698514	4d2322	VJT716	Malta	1781698513	1781698513	28.926	34.3443	14935.2	false	258.4	123.88	0.0	NULL	15544.8	5260	false	0
2026-06-17T11:40:36.712189+00:00	1781696433	4d2322	VJT716	Malta	1781696432	1781696432	23.9054	36.9215	14935.2	false	257.55	120.36	0.0	NULL	15453.36	5260	false	0
2026-06-17T11:40:36.712189+00:00	1781696433	e497e2	PSFMV	Brazil	1781696431	1781696432	-105.0222	31.7179	14317.98	false	222.03	282.72	0.0	NULL	14988.54	3776	false	0
2026-06-17T11:40:36.712189+00:00	1781696433	a0574c	N1209G	United States	NULL	1781696431	NULL	NULL	14325.6	false	2111.56	240.26	8.45	NULL	14874.24	3233	false	0



origin_country	count
Portugal	26

8) The producer was run multiple times to simulate the streamer.



2. Questions and Results Achieved

The following are the answers to all the questions posed in the Exercise and how they were found:

Q1. Choose with your team member a deployment option and justify your choice. Note that during the deployment you might face issues (e.g., network connectivity, conflicting ports) and you might change your option, this is acceptable, but in such case, please document the reason to not follow the plan.
We chose option 2, the reason was to facilitate our life. This option was by far the easiest because all containers will be running in the same environment using a single Docker Compose configuration which meant communication was trivial.

Q2. Which Docker Compose (or other) files you have used (present them as an annex to this report). See **Annex** below.

Q3. How can your guarantee that the deployment is successful? How can you test the communication between Spark and Kafka? Document the steps you follow to perform this step. You might need to develop Python code.
The Docker ps command showed that all containers were running. the UI for Spark and Kafka shows everything is running, we did not write separate test code; instead, we validated the deployment using the final implementation which can be seen in **Annex** below.

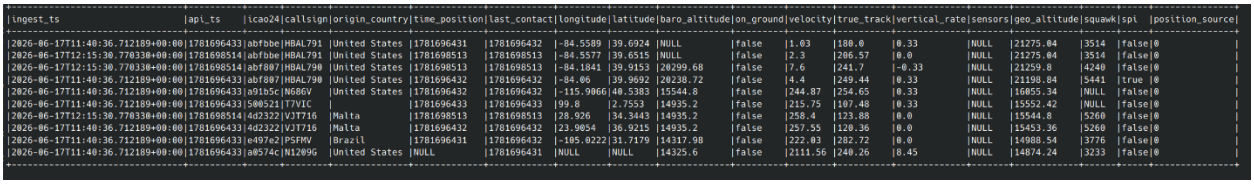
Q4. Which modifications have you performed in the producer? Explain them in brief words (attach the file as an annex to this report).
we modified the Producer.py source code to read the API and publish the retrieved data to a Kafka topic. However, we needed to add a batching process as the unbatched data was too large. For the full source code, see **Annex** below.

Q5. Which modifications have you performed in the consumer? Explain them in brief words (attach the file as an annex to this report). It is expected that the consumer performs the same functionalities has the previous exercise.

Multiple changes were required for the consumer; these are described in detail in **Annex** 3. However, the highlights are: Updated the code to read from the Kafka topic, Configured the correct settings to connect to Kafka and Added queries to retrieve requested information.

Q6. What are the top 10 flights with a higher altitude (consider the geometric altitude, as it corresponds to the one, we are used to)? You need to provide the code with this functionality.

```
def show_top_10(batch_df, batch_id):
    (
        batch_df
        .filter(col("geo_altitude").isNotNull())
        .orderBy(col("geo_altitude").desc())
        .limit(10)
        .show(truncate=False)
    )

top_10_highest_altitude_flights_query = (
    parsed_states.writeStream
    .foreachBatch(show_top_10)
    .outputMode("append")
    #.option("checkpointLocation", "checkpoints/new_top_10_highest_altitude_flights")
    .start()
)
```



ingest_ts	api_ts	icao24	callsign	origin_country	time_position	last_contact	longitude	latitude	baro_altitude	on_ground	velocity	true_track	vertical_rate	sensors	geo_altitude	squawk	spi	position_source
2026-06-17T11:40:36.712189+00:00	1781696433	abfbbe	HBAL791	United States	1781696431	1781696432	-84.5589	39.6924	NULL	false	1.03	180.0	0.33	NULL	21275.04	3514	false	0
2026-06-17T12:15:30.770330+00:00	1781698514	abfbbe	HBAL791	United States	1781698513	1781698513	-84.5577	39.6515	NULL	false	2.3	206.57	0.0	NULL	21275.04	3514	false	0
2026-06-17T12:15:30.770330+00:00	1781698514	abf807	HBAL790	United States	1781698513	1781698513	-84.1841	39.9153	20299.68	false	7.6	241.7	-0.33	NULL	21259.8	4240	false	0
2026-06-17T11:40:36.712189+00:00	1781696433	abf807	HBAL790	United States	1781696432	1781696432	-84.06	39.9692	20238.72	false	4.4	249.44	0.33	NULL	21198.84	5441	true	0
2026-06-17T11:40:36.712189+00:00	1781696433	a91b5c	N686V	United States	1781696432	1781696432	-115.9066	40.5383	15544.8	false	244.87	254.65	0.33	NULL	16055.34	NULL	false	0
2026-06-17T11:40:36.712189+00:00	1781696433	500521	T7VIC		1781696433	1781696433	99.8	2.7553	14935.2	false	215.75	107.48	0.33	NULL	15552.42	NULL	false	0
2026-06-17T12:15:30.770330+00:00	1781698514	4d2322	VJT716	Malta	1781698513	1781698513	28.926	34.3443	14935.2	false	258.4	123.88	0.0	NULL	15544.8	5260	false	0
2026-06-17T11:40:36.712189+00:00	1781696433	4d2322	VJT716	Malta	1781696432	1781696432	23.9854	36.9215	14935.2	false	257.55	120.36	0.0	NULL	15453.36	5260	false	0
2026-06-17T11:40:36.712189+00:00	1781696433	e497e2	PSFMV	Brazil	1781696431	1781696432	-105.0222	31.7179	14317.98	false	222.03	282.72	0.0	NULL	14988.54	3776	false	0
2026-06-17T11:40:36.712189+00:00	1781696433	a0574c	N1209G	United States	NULL	1781696431	NULL	NULL	14325.6	false	2111.56	240.26	8.45	NULL	14874.24	3233	false	0

Q7. Count the number of flights of Portugal on the ground?

```
# count the number of flights of Portugal on the ground.
portugal_flights_on_ground = (
    parsed_states
    .filter((col("origin_country") == "Portugal") & (col("on_ground") == True))
    .groupBy("origin_country")
    .count()
)

portugal_flights_on_ground_query = (
    portugal_flights_on_ground.writeStream
    .format("console")
    .outputMode("complete")
    #.option("checkpointLocation", "checkpoints/new_portugal_flights_on_ground")
    .start()
)
query.awaitTermination()
```

```
-------------------------------------------
Batch: 6
-------------------------------------------
+--------------+-----+
|origin_country|count|
+--------------+-----+
|      Portugal|   53|
+--------------+-----+
```

Q8. Which model would be chosen to perform the prediction of the barometric altitude? Justify your answer.

We would choose **Linear Regression** from Spark MLlib.

The target variable, **barometric altitude**, is numeric, and the input variable, **geometric altitude**, is also numeric. So this is a **regression problem**, not classification. A linear regression model is appropriate because barometric altitude and geometric altitude are expected to have a strong continuous relationship. Section 4.5 specifically asks to predict barometric altitude from geometric altitude using Spark MLlib. Spark MLlib supports linear regression for predicting numeric labels from feature vectors. (Spark.apache.org)

Q9. How could you use it in Spark? Document how you could train it, and how you could integrate it with your stream processing.

In Spark, the data would be prepared with:

```
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import LinearRegression
```

Training idea:

```
training_df = parsed_states.select(
    col("geo_altitude"),
    col("baro_altitude").alias("label")
).dropna()

assembler = VectorAssembler(
    inputCols=["geo_altitude"],
    outputCol="features"
)

train_ready = assembler.transform(training_df)

lr = LinearRegression(
    featuresCol="features",
    labelCol="label"
)
model = lr.fit(train_ready)
```

To integrate with stream processing, train the model on historical/static Kafka data or saved flight data first. Then, inside the streaming consumer, transform each incoming batch with the same VectorAssembler and apply: `predictions = model.transform(stream_ready)`
For Structured Streaming, the usual approach is **train offline, predict online**. The stream reads Kafka records, parses altitude fields, creates a features column from geo_altitude, then applies the saved model to produce a predicted baro_altitude. Spark's streaming model works in micro-batches, so this can also be applied using foreachBatch.

3.Results achieved

- All questions were answered to a high standard.
- Consumer and producer modified as requested in the task.
- Verified that data was successfully sent to Kafka, received by the consumer, and ingested by Spark.
- Research completed for MLlib.

4.Lessons Learned

A couple of important lessons were learned while completing this exercise:

- We needed to modify the producer to send batches in smaller messages rather than one big message because the payload exceeded the maximum message size.
- When merging multiple Docker Compose files, it is important to verify port assignments are being used as conflicts can arise. In this case both the Kafka UI and the Spark master UI were both assigned 8080, so to get this to work we moved the Kafka UI to port 8082.
- To communicate correctly between Kafka and Spark we needed to use following command (as mentioned in the slide 17)

```
spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.13:4.0.0
--verbose kafkaStreamingStats.py | tee example2.log
```

  Unfortunately, this did not work and produced a java error. After research it was discovered that the Kafka version must match the installed Spark version exactly so 4.0.0 has to change to 4.1.2 in our case.

- Spark submit generates log lines output at the INFO level by default. This means it was almost impossible to see the actual output vs these INFO messages. Using the following code in the consumer which suppressed the INFO messages and made the application output easier to read.

```
# suppress all logs except for errors
spark.sparkContext.setLogLevel("ERROR")
```

Annexes

Note: All relevant files have been attached.

Annex 1 – Docker Compose

```
# Merged Docker Compose file: Kafka + Kafka UI + Apache Spark
# Spark containers can reach Kafka using: kafka:29092
# From your host machine, Kafka is reachable on: localhost:9092

services:
  kafka:
    container_name: kafka
    image: apache/kafka:4.3.0
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: "CONTROL-
LER:PLAINTEXT,PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT"
      KAFKA_ADVERTISED_LISTENERS: "PLAINTEXT://kafka:29092,PLAINTEXT_HOST://lo-
calhost:9092"
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
      KAFKA_PROCESS_ROLES: "broker,controller"
      KAFKA_NODE_ID: 1
      KAFKA_CONTROLLER_QUORUM_VOTERS: "1@kafka:29093"
      KAFKA_LISTENERS: "PLAINTEXT://kafka:29092,CONTROL-
LER://kafka:29093,PLAINTEXT_HOST://0.0.0.0:9092"
      KAFKA_INTER_BROKER_LISTENER_NAME: "PLAINTEXT"
      KAFKA_CONTROLLER_LISTENER_NAMES: "CONTROLLER"
      KAFKA_LOG_DIRS: "/tmp/kafka-combined-logs"
      CLUSTER_ID: "MkU3OEVBNTcwNTJENDM2Qk"
    volumes:
      - ./kafka-config:/opt/kafka/config/
      - ./kafka-data:/tmp/kafka-combined-logs
    networks:
      - aii-network

  kafbat-ui:
    container_name: kafbat-ui
    image: ghcr.io/kafbat/kafka-ui:latest
```

```
    depends_on:
      - kafka
    ports:
      - "8082:8080"
    environment:
      DYNAMIC_CONFIG_ENABLED: "true"
      #DYNAMIC_CONFIG_ENABLED: "true"
      SWAGGER_UI_ENABLED: "true"
      KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:29092
      KAFKA_CLUSTERS_0_NAME: myKafka
    #volumes:
      #- ./kafka.ui-config.yml:/etc/kafkaui/dynamic_config.yaml
    networks:
      - aii-network

  spark-master:
    image: spark-custom:latest
    container_name: spark-master
    ports:
      - "7077:7077"
      - "8080:8080"
      - "18080:18080"    # History Server UI
      - "4040:4040"      # Spark Application UI
      - "9999:9999"      # Custom port for additional services
    environment:
      - SPARK_MODE=master
      - SPARK_CONF_spark_hadoop_fs_viewfs_impl_disable=true
      - SPARK_CONF_spark_hadoop_fs_AbstractFileSystem_viewfs_impl=org.apache.ha-
doop.fs.viewfs.ViewFs
      - SPARK_RPC_AUTHENTICATION_ENABLED=no
      - SPARK_RPC_ENCRYPTION_ENABLED=no
      - SPARK_LOCAL_STORAGE_ENCRYPTION_ENABLED=no
    command: /opt/spark/bin/spark-class org.apache.spark.deploy.master.Master
    networks:
      - aii-network

  spark-worker-1:
    #image: apache/spark:latest
    image: spark-custom:latest
    container_name: spark-worker-1
    ports:
      - "8081:8081"
      - "4041:4040"      # Spark Application UI
      - "9998:9999"      # Custom port for additional services
    environment:
```

```
      - SPARK_MODE=worker
      - SPARK_MASTER_URL=spark://spark-master:7077
      - SPARK_WORKER_MEMORY=2G
      - SPARK_WORKER_CORES=2
      - SPARK_CONF_spark_hadoop_fs_viewfs_impl_disable=true
      - SPARK_CONF_spark_hadoop_fs_AbstractFileSystem_viewfs_impl=org.apache.ha-
doop.fs.viewfs.ViewFs
    command: /opt/spark/bin/spark-class org.apache.spark.deploy.worker.Worker
spark://spark-master:7077
    depends_on:
      - spark-master
    networks:
      - aii-network
    volumes:
      - ./:/opt/spark/mycodeStreaming

networks:
  aii-network:
    driver: bridge
```

Annex 2 – Producer

```
'''
Python Kafka Producer Example
'''

from kafka import KafkaProducer
import json
from time import sleep

import os
import json
import requests
from datetime import datetime, timezone

API_URL = "https://opensky-network.org/api/states/all"
OUT_DIR = "data/opensky_raw"

def fetch_and_save():
    os.makedirs(OUT_DIR, exist_ok=True)
    r = requests.get(API_URL, timeout=30)
    r.raise_for_status()
    payload = r.json()
```

```
    record = {
        "ingest_ts": datetime.now(timezone.utc).isoformat(),
        "api_ts": payload.get("time"),
        "states": payload.get("states")
    }
    filename = f"opensky_{datetime.now(time-
zone.utc).strftime('%Y%m%dT%H%M%S%fZ')}.json"
    path = os.path.join(OUT_DIR, filename)
    with open(path, "w", encoding="utf-8") as f:
        json.dump(record, f)

def get_messages_from_api():
    r = requests.get(API_URL, timeout=30)
    r.raise_for_status()
    payload = r.json()
    record = {
        "ingest_ts": datetime.now(timezone.utc).isoformat(),
        "api_ts": payload.get("time"),
        "states": payload.get("states")
    }
    return record

def json_serializer(data):
    return json.dumps(data).encode('utf-8')

producer = KafkaProducer(
    bootstrap_servers=['localhost:9092'],
    value_serializer=json_serializer,
    #max_request_size=10485760,
    #batch_size=16384,  # 16k
    #linger_ms=50
)

def produce_messages(topic, messages):
    for message in messages:
        producer.send(topic, value=message)
        print(f"Produced messages messgage to topic {topic}")
        #sleep(1)  # Simulate some delay between messages
    producer.flush()  # Ensure all messages are sent

if __name__ == "__main__":
    topic = 'myTopic' # Change to your topic name
    # loop over the API and produce messages and call every second
    while True:
        messages = [get_messages_from_api()]
```

```
        # messages is too big to be sent in one message, so we need to split it
into smaller messages
        # we can split the states into smaller messages and send them separately
        states = messages[0]['states']
        batch_size = 100  # Number of states per message
        messages_to_send = []
        for i in range(0, len(states), batch_size):
            batch = states[i:i + batch_size]
            message = {
                "ingest_ts": messages[0]['ingest_ts'],
                "api_ts": messages[0]['api_ts'],
                "states": batch
            }
            messages_to_send.append(message)
            print(f"Prepared message with {len(batch)} states to send.")
        produce_messages(topic, messages_to_send)
        print("All messages produced.")
        exit()
        sleep(1)  # Wait for 1 second before fetching new messages
```

Annex 3 – Consumer

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, LongType, Ar-
rayType, BooleanType, DoubleType
from pyspark.sql.functions import col, explode, from_json

STATE_FIELDS = [
    "icao24",
    "callsign",
    "origin_country",
    "time_position",
    "last_contact",
    "longitude",
    "latitude",
    "baro_altitude",
    "on_ground",
    "velocity",
    "true_track",
    "vertical_rate",
    "sensors",
    "geo_altitude",
    "squawk",
    "spi",
```

```
    "position_source",
]

NUMERIC_CASTS = {
    "time_position": LongType(),
    "last_contact": LongType(),
    "longitude": DoubleType(),
    "latitude": DoubleType(),
    "baro_altitude": DoubleType(),
    "on_ground": BooleanType(),
    "velocity": DoubleType(),
    "true_track": DoubleType(),
    "vertical_rate": DoubleType(),
    "geo_altitude": DoubleType(),
}

spark = (
    SparkSession.builder.appName("OpenSkyStream")
    .config("spark.jars.packages", "org.apache.spark:spark-sql-kafka-0-
10_2.13:4.1.2")
    .getOrCreate()
)

# suppress all logs except for errors
spark.sparkContext.setLogLevel("ERROR")

# 2. Read from Kafka
topicKafka = "myTopic" # Replace with your Kafka topic name

schema = StructType([
    StructField("ingest_ts", StringType(), True),
    StructField("api_ts", LongType(), True),
    StructField("states", ArrayType(ArrayType(StringType(), True), True), True),
])

stream_df = (
    spark.readStream.format("kafka")
    #.option("kafka.bootstrap.servers", "localhost:9092")
    #.option("kafka.bootstrap.servers", "172.17.0.1:9092")
    .option("kafka.bootstrap.servers", "kafka:29092")
    .option("subscribe", topicKafka)
    .option("startingOffsets", "earliest")
    .load()
)
```

```
parsed_payload = stream_df.select(
    from_json(col("value").cast("string"), schema).alias("payload")
).select("payload.*")

parsed_states = (
    parsed_payload
    .withColumn("state", explode(col("states")))
    .select(
        "ingest_ts",
        "api_ts",
        *[
            (
                col("state").getItem(idx).cast(NUMERIC_CASTS[field])
                if field in NUMERIC_CASTS
                else col("state").getItem(idx)
            ).alias(field)
            for idx, field in enumerate(STATE_FIELDS)
        ]
    )
)

parsed_states.printSchema()

# What are the top 10 flights with a higher altitude (consider the geometric
# altitude, as it corresponds to the one, we are used to). You need to provide
the code
# with this functionality.
top_10_highest_altitude_flights = (
    parsed_states
    .filter(col("geo_altitude").isNotNull())
    .orderBy(col("geo_altitude").desc())
)

query = (
    parsed_states.writeStream
    .format("console")
    .outputMode("append")
    #.option("checkpointLocation", "checkpoints/opensky_stream")
    .start()
)

print("doing query")
def show_top_10(batch_df, batch_id):
    (
        batch_df
```

```
        .filter(col("geo_altitude").isNotNull())
        .orderBy(col("geo_altitude").desc())
        .limit(10)
        .show(truncate=False)
    )

top_10_highest_altitude_flights_query = (
    parsed_states.writeStream
    .foreachBatch(show_top_10)
    .outputMode("append")
    #.option("checkpointLocation", "checkpoints/new_top_10_highest_alti-
tude_flights")
    .start()
)

# ount the number of flights of Portugal on the ground.
portugal_flights_on_ground = (
    parsed_states
    .filter((col("origin_country") == "Portugal") & (col("on_ground") == True))
    .groupBy("origin_country")
    .count()
)

portugal_flights_on_ground_query = (
    portugal_flights_on_ground.writeStream
    .format("console")
    .outputMode("complete")
    #.option("checkpointLocation", "checkpoints/new_portugal_flights_on_ground")
    .start()
)
query.awaitTermination()
```

Remark: ChatGPT assisted only with report wording and minor coding corrections and played no role in the core analysis or results.